

*"Same code, every environment --
configuration should change, never the
codebase."*

MODULE 26

Spring Configuration, Profiles and Environments

Manage Spring Boot application configuration effectively across development, test, and production using profiles, externalized properties, and secure secrets handling.





MODULE 26 OVERVIEW

Manage Spring Boot application configuration effectively across development, test, and production using profiles, externalized properties, and secure secrets handling.

Configuration -- not code -- should change between environments. This module covers YAML/properties, Spring Profiles, property-source precedence, and secrets handling, then locks down Northstar CRM's dev/test/prod environments.

DURATION & LAB



3 Hours Theory

application.yml vs application.properties, Spring Profiles, property-source precedence, and @ConfigurationProperties.



90 Minutes Lab

Northstar CRM Environments: build dev/test/prod profile files with environment-variable secrets.



Scenario

Dev settings leaking into production (open H2 console, blank DB password) are unacceptable -- prod must fail fast.

MODULE ARC



Understand Precedence

CLI args beat environment variables beat profile YAML beat application.yml beat code defaults.



Master Profiles & Secrets

Activate profiles two ways; bind @ConfigurationProperties; never hard-code secrets.



Build the Lab

Convert Northstar CRM to application.yml with dev/test/prod profiles and environment-variable credentials.

KEY TAKEAWAY Total time: 3 hours theory + 90 minutes hands-on lab. Good configuration leads to flexible applications; secure configuration leads to trustworthy ones.

WHY ENVIRONMENT MANAGEMENT MATTERS

Applications run in different environments -- development, testing, staging, and production -- each with unique requirements and constraints.

Effective environment management ensures your application is flexible, secure, and reliable across the entire delivery lifecycle.

THE CHALLENGE WITHOUT PROPER ENVIRONMENT MANAGEMENT

Hard-coded Values

Security Risks

Inconsistent Behavior

Deployment Delays

Higher Operational Cost

THE BENEFITS OF EFFECTIVE ENVIRONMENT MANAGEMENT



Consistency

Maintain consistent behavior across all environments; reduce environment-specific issues.



Security

Keep sensitive data out of source code with secure, externalized configuration.



Flexibility

Switch configurations across environments without any code changes.



Faster Delivery

Simplify builds and promote code through environments with confidence.



Maintainability

Centralize and organize configuration; easier to manage as applications grow.



Team Productivity

Developers focus on features, not environment setup; fewer errors, more value.

KEY TAKEAWAY Environment management is not just about configuration -- it's about building applications that are portable, secure, and enterprise-ready.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to effectively manage Spring Boot application configuration across multiple environments using profiles and externalized properties.

- 1. Understand Configuration Fundamentals** — explain the role of configuration in Spring Boot applications and compare application.properties and application.yml.
- 2. Work with YAML Configuration** — use YAML syntax and structure to organize and manage application configuration effectively.
- 3. Use Spring Profiles** — create and activate profiles for development, test, and production environments.
- 4. Manage Environment Configuration** — leverage environment variables, command-line arguments, and external property sources to externalize configuration.
- 5. Secure Sensitive Information** — apply best practices to protect sensitive data and explore basic secrets management strategies.
- 6. Apply Enterprise Configuration Practices** — implement configuration strategies across environments and follow best practices for maintainable, secure deployments.

KEY TAKEAWAY Master Spring Profiles and configuration to build flexible, secure, and environment-aware applications.

CONFIGURATION MANAGEMENT OVERVIEW (1 OF 2)

Configuration management is the practice of managing application settings and externalizing them from code so applications can adapt to different environments.

WHY IT MATTERS



Flexibility

Change settings without modifying or rebuilding code.



Reliability

Ensure consistent behavior across environments.



Scalability

Support multiple environments and deployments.



Security

Protect sensitive information using secure practices.



Cost Efficiency

Reduce errors and downtime from misconfiguration.

KEY PRINCIPLES

Externalize Configuration — store configuration outside the application so it can be changed depending on the environment.

Environment Awareness — use environment-specific settings for development, test, staging, and production.

Separation of Concerns — keep configuration separate from business logic and source code.

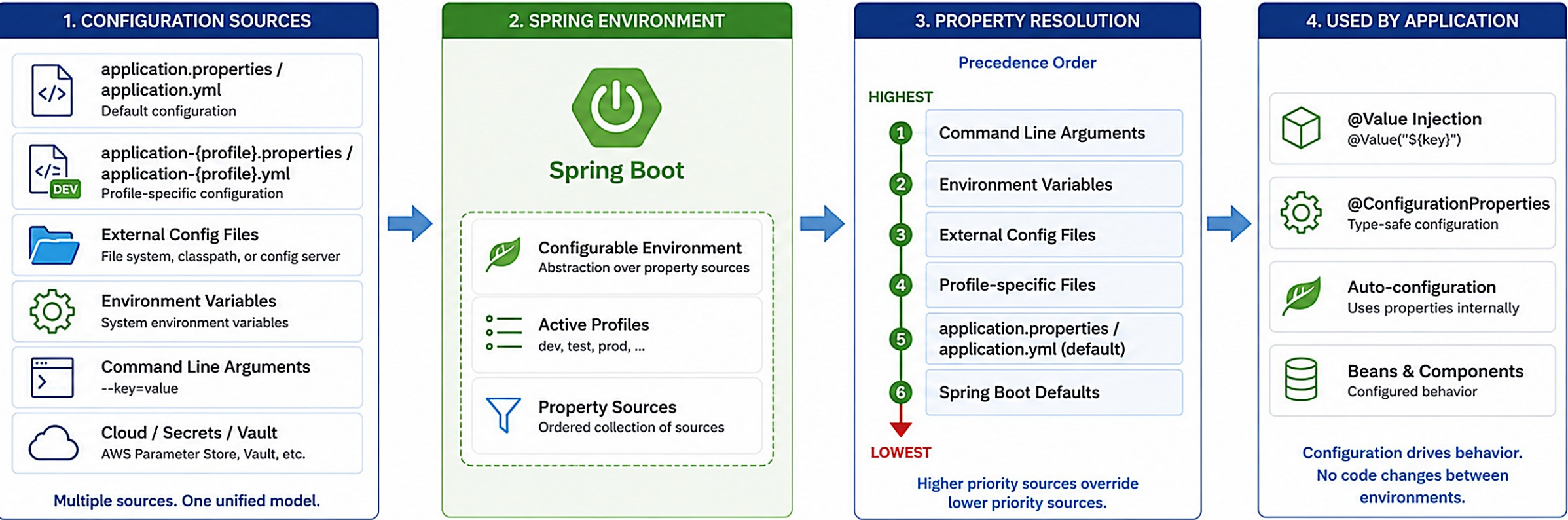
Centralized and Consistent — manage configuration in a consistent and organized way across services and teams.

Secure and Governed — protect sensitive data and follow governance for configuration changes.

KEY TAKEAWAY Configuration management is the practice of managing application settings and externalizing them from code so applications can adapt to different environments.

Configuration Management Overview

Spring Boot provides a powerful and flexible way to externalize configuration, manage different environments using profiles, and resolve properties in a defined order of precedence.



Externalized Configuration
Keep config separate from code

Environment Flexibility
Different settings for different environments

Centralized Management
Consistent and maintainable configuration

Secure Configuration
Protect sensitive information

Cloud & Enterprise Ready
Integrates with modern infrastructure

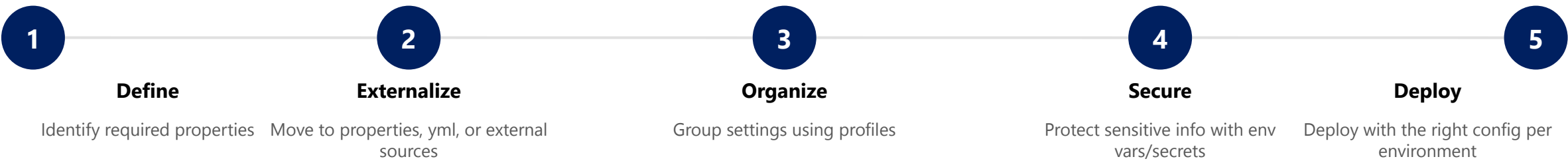
Goal
Right configuration, in the right place, for the right environment.



CONFIGURATION MANAGEMENT OVERVIEW (2 OF 2)

The configuration management flow, and the typical sources configuration comes from in a Spring Boot application.

CONFIGURATION MANAGEMENT FLOW



TYPICAL CONFIGURATION SOURCES

Source	Description
application.properties / application.yml	Default configuration files packaged with the application.
Environment Variables	Environment-specific values provided by the OS or container platform.
Command-line Arguments	Override properties at startup using --key=value.
External Configuration Servers	Centralize configuration using systems like Spring Cloud Config.
Secrets Management Systems	Store and retrieve sensitive information securely (e.g. AWS Secrets Manager).

KEY TAKEAWAY Goal: deliver the right configuration to the right environment at the right time.

APPLICATION.PROPERTIES

The default Spring Boot configuration file, using a simple key-value format.

Example: application.properties

```
# Application identity
spring.application.name=DemoApp

# Server configuration
server.port=8080
server.servlet.context-path=/api

# Database configuration
spring.datasource.url=jdbc:mysql://localhost:3306/demo
spring.datasource.username=dbuser
spring.datasource.password=dbpassword

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update

# Logging
logging.level.root=INFO
```

KEY CHARACTERISTICS

Simple Key-Value Format — uses dot notation for hierarchical properties.

Auto Loaded by Spring Boot — automatically detected from src/main/resources.

Supports Profiles — override properties per environment (application-dev.properties).

Externalizable — can be overridden by environment variables or command-line args.

COMMON USE CASES

Database Connections

Server/Port Settings

Logging Levels

Feature Flags

KEY TAKEAWAY Use application.properties for simplicity; use application.yml when you need more complex or structured configuration.

APPLICATION.YML

A YAML-based alternative for cleaner, hierarchical configuration.

Example: application.yml

```
spring:
  application:
    name: DemoApp
server:
  port: 8080
  servlet:
    context-path: /api
datasource:
  url: jdbc:mysql://localhost:3306/demo
  username: dbuser
  password: dbpassword
jpa:
  hibernate:
    ddl-auto: update
logging:
  level:
    root: INFO
```

YAML vs PROPERTIES

application.properties	application.yml
Flat key-value format	Hierarchical structure
Harder to read at scale	More readable
Repetitive keys	Less repetition

KEY BENEFITS

- Hierarchical Structure** — natural nesting of related properties.
- Less Duplication** — avoids repeating long property name prefixes.
- Profile Support** — easy to define environment-specific configuration.

KEY TAKEAWAY Spring Boot automatically detects and loads application.yml if present in src/main/resources, giving it the same priority as application.properties.



YAML SYNTAX FUNDAMENTALS

YAML is a human-friendly data serialization format that uses indentation and structure instead of brackets and braces.

CORE SYNTAX RULES

Indentation Matters — YAML uses spaces (never tabs) to define structure and nesting.

Key: Value Pairs — keys and values are separated by a colon and a space.

Lists — defined using a hyphen (-) followed by a space.

Strings — can be plain, single-quoted, or double-quoted.

Comments — use # to add a comment; everything after # is ignored.

Example: application.yml

```
spring:
  application:
    name: DemoApp
  server:
    port: 8080
  datasource:
    url: jdbc:mysql://host/db
  features:
    - payments
    - notifications
  logging:
    level:
      root: INFO
      com.example: DEBUG
```

DATA TYPES IN YAML

Type	Example
String	name: DemoApp
Number	port: 8080
Boolean	enabled: true
Null	host: null
List	- payments
Map	database: {...}

BEST PRACTICES

- 2-space Indentation
- Avoid Tabs
- Meaningful Key Names
- Quote Strings Only When Needed

KEY TAKEAWAY YAML is case-sensitive. Correct indentation ensures your configuration is loaded properly.

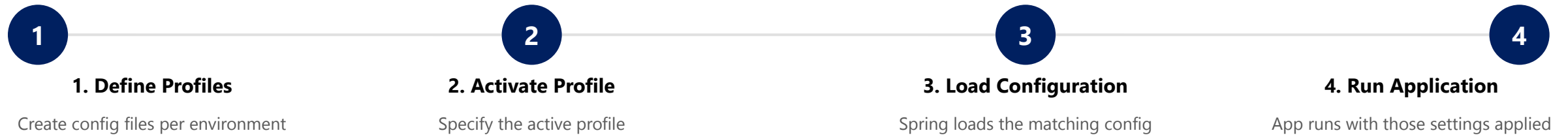


WHAT ARE SPRING PROFILES? (1 OF 2)

Spring Profiles allow you to define different configurations for different environments and activate the appropriate configuration at runtime.

Profiles help you build flexible applications that can adapt to environment-specific settings without changing the code.

HOW IT WORKS



KEY BENEFITS

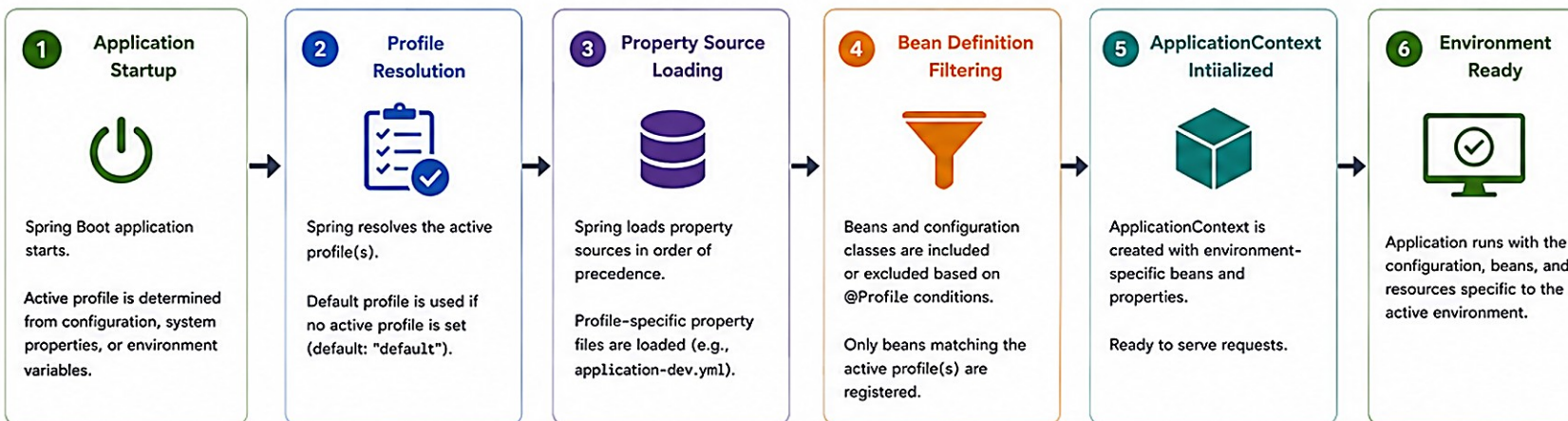
- Environment-specific configuration with no code changes to switch environments.
- Cleaner, more maintainable codebase, and easier deployment across environments.
- Supports dev, test, staging, prod, and more.

KEY TAKEAWAY Spring Profiles let you switch entire configurations at runtime, without touching a single line of code.



Spring Profiles and Environment Flow

Manage different environments (dev, test, stage, prod) with Spring Profiles



HOW PROFILES ARE ACTIVATED

- Command Line**
--spring.profiles.active=dev
- System Property**
-Dspring.profiles.active=dev
- Environment Variable**
SPRING_PROFILES_ACTIVE=dev
- application.properties / .yml**
spring.profiles.active=dev

@Profile USAGE

```
@Profile("dev")
@Configuration
public class DevConfig {
    // dev specific beans
}

@Profile({"stage", "prod"})
@Service
public class AuditService {
    // used in stage and prod
}
```

DEFAULT PROFILE

If no active profile is set, Spring uses the default profile: "default".

PROFILE-SPECIFIC CONFIGURATION FILES

Base Configuration (Always Loaded)

```
application.yml

app:
  name: MyApp
  logging:
    level:
      root: INFO
```

Profile Specific (Loaded when active)

```
application-dev.yml

server:
  port: 8080
datasource:
  url: jdbc:h2:mem:devdb
logging:
  level:
    root: DEBUG
```

```
application-test.yml

server:
  port: 8081
datasource:
  url: jdbc:h2:mem:testdb
logging:
  level:
    root: INFO
```

```
application-stage.yml

server:
  port: 8082
datasource:
  url: jdbc:postgresql://stage-db/mydb
logging:
  level:
    root: WARN
```

```
application-prod.yml

server:
  port: 80
datasource:
  url: jdbc:postgresql://prod-db/mydb
logging:
  level:
    root: ERROR
```

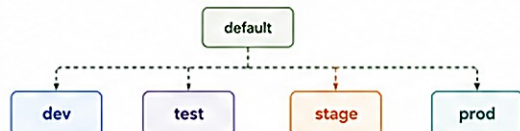
Active Profile Example

Active Profile: dev

Effective Configuration

```
server.port=8080
app.name=MyApp
datasource.url=jdbc:h2:mem:devdb
logging.level.root=DEBUG
...
```

PROFILE HIERARCHY



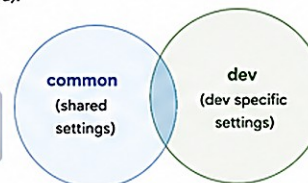
Properties from specific profile override those from the default profile.
Custom hierarchies can be defined using spring.profiles.include.

MULTIPLE ACTIVE PROFILES

You can activate multiple profiles (comma separated).
Order of precedence (last one wins for conflicts).

Example

```
--spring.profiles.active=common,dev
# dev properties override common if same key exists
```



KEY BENEFITS

- Environment Isolation**
Keep configurations separate for each environment.
- Easy Configuration Management**
Change settings without changing code.
- Safe Deployments**
Different settings for dev, test, stage, and prod.
- Scalability**
Supports complex applications with multiple environments.



Best Practice:

Use profiles to externalize environment-specific configuration and keep your application portable and maintainable.

Spring Profiles

Spring Profiles let you define and activate different sets of configuration for different environments.

What are Spring Profiles?



Profiles allow you to isolate environment-specific configuration.



Only the active profile's properties and beans are loaded.



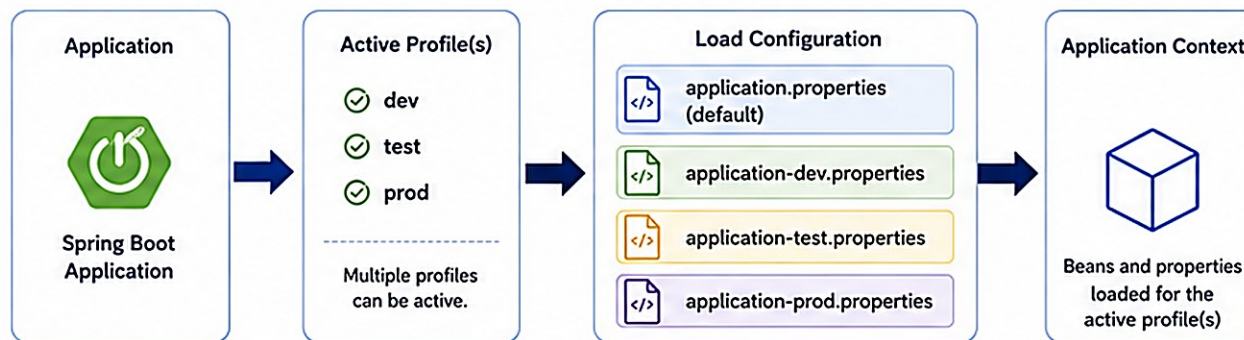
Helps manage different environments using the same codebase.



Key Benefit

Cleaner configuration, safer deployments, and easier environment management.

How Profiles Work



Example



Active Profile: dev

- application.properties
- application-dev.properties



Active Profile: test

- application.properties
- application-test.properties



Active Profile: prod

- application.properties
- application-prod.properties

Built-in & Custom Profiles

Built-in Profiles



default

Used when no other profile is active.



test

Convention for test environment.



prod

Convention for production environment.

Custom Profiles



Create any profile that fits your needs.

Examples: dev, qa, staging, uat, local

Profile Activation

1. application.properties / application.yml



```
spring.profiles.active=dev
# or in YAML
spring:
  profiles:
    active: dev
```

OR



2. Command Line Argument

```
--spring.profiles.active=dev
java -jar app.jar \
  --spring.profiles.active=dev
```

OR



3. Environment Variable

```
SPRING_PROFILES_ACTIVE=dev
# Linux / Mac
export SPRING_PROFILES_ACTIVE=dev
# Windows (Command Prompt)
set SPRING_PROFILES_ACTIVE=dev
```

OR



4. Programmatically

```
SpringApplication app =
  new SpringApplication(App.class);
app.setAdditionalProfiles("dev");
app.run(args);
```



Tip: You can activate multiple profiles by comma-separated values (e.g., dev,common) for shared configuration.



Note: Profile-specific files override the default application.properties.

WHAT ARE SPRING PROFILES? (2 OF 2)

Profile configuration sources in order of precedence, profile-specific file naming, and how to activate a profile.

PROFILE CONFIGURATION SOURCES (IN ORDER OF PRECEDENCE)

- 1. Command-line arguments
- 2. Java System properties
- 3. OS environment variables
- 4. application.properties / application.yml
- 5. Default profile (when no profile is active)

EXAMPLE: PROFILE-SPECIFIC FILES

application.yml — common configuration shared by every environment.

application-dev.yml — development-specific settings.

application-test.yml — test-specific settings.

application-prod.yml — production-specific settings.

Example Activation

```
--spring.profiles.active=dev
-Dspring.profiles.active=test
SPRING_PROFILES_ACTIVE=prod

# Default profile name: default
```

IMPORTANT

Only one profile is normally active at a time (profiles can be combined via profile groups). If no profile is specified, Spring uses the default profile.

KEY TAKEAWAY src/main/resources/application-{profile}.yml is automatically loaded by Spring Boot when that profile is active.

DEVELOPMENT PROFILE

Used during local development -- debug logging, in-memory databases, relaxed security.

Example: application-dev.yml

```
spring:
  profiles:
    active: dev
  datasource:
    url: jdbc:h2:mem:devdb;DB_CLOSE_DELAY=-1
    driver-class-name: org.h2.Driver
  h2:
    console:
      enabled: true
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  logging:
    level:
      root: DEBUG
```

KEY CHARACTERISTICS

- Optimized for local development and fast feedback.
- Uses in-memory databases (e.g. H2) and mock services.
- Enables detailed logging and debugging.
- Relaxed security for easier development.
- May include sample data and developer tools.

HOW TO ACTIVATE

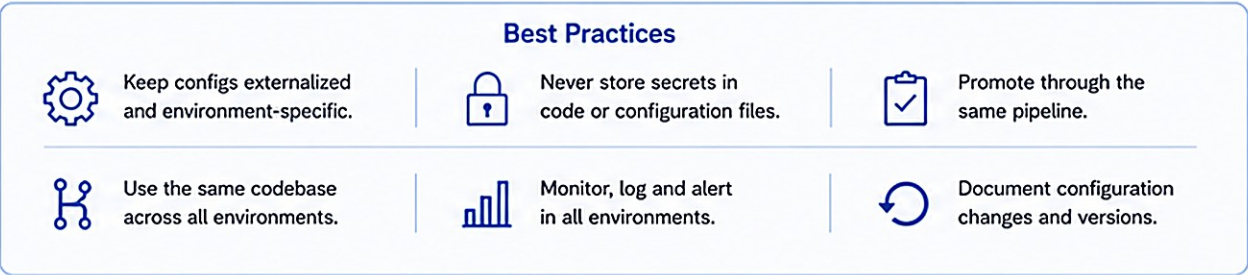
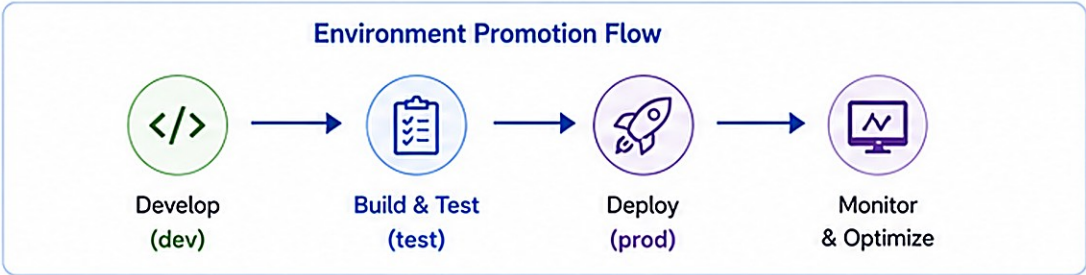
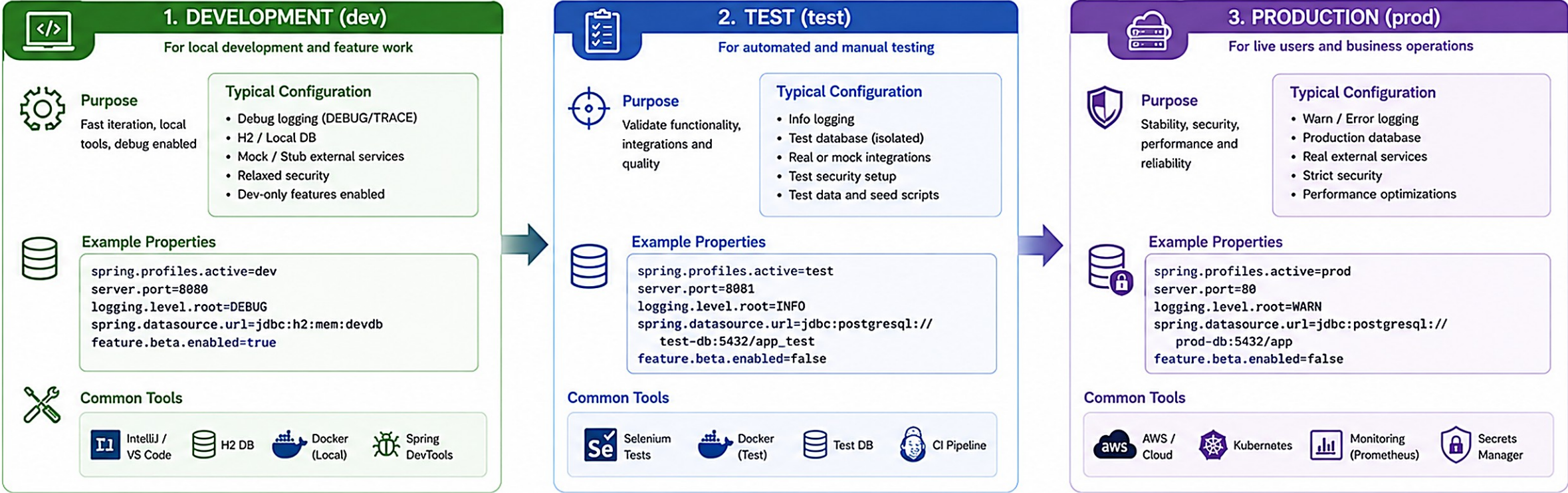
-Dspring.profiles.active=dev

SPRING_PROFILES_ACTIVE=dev

KEY TAKEAWAY Never use the dev profile in production. It is meant only for local developer environments.

Spring Profiles: Development, Test, Production

Use Spring Profiles to keep configuration, beans, and resources environment-specific and promote safely from development to production.



Key Takeaway: Profiles let you adapt the same application to different environments with the right configuration, improving reliability and reducing risk.

TEST PROFILE

Used for automated testing -- a stable, isolated environment for fast, repeatable tests.

Example: application-test.yml

```
spring:
  profiles:
    active: test
  datasource:
    url: jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1
    driver-class-name: org.h2.Driver
  h2:
    console:
      enabled: false
  jpa:
    hibernate:
      ddl-auto: create-drop
    show-sql: false
  logging:
    level:
      root: WARN
```

KEY CHARACTERISTICS

- Uses in-memory databases (e.g. H2) created fresh for each test run.
- Loads test data and mocks external dependencies.
- Optimized for automated tests -- speed and repeatability.
- Reduced security complexity for testing ease.
- Ensures consistent test execution across environments.

HOW TO ACTIVATE

-Dspring.profiles.active=test

SPRING_PROFILES_ACTIVE=test

KEY TAKEAWAY Never use the test profile in staging or production. It is designed only for testing environments.

Test Profile

Isolated, Fast, and Reliable Testing with Spring Profiles



The **Test Profile** provides a dedicated configuration for running tests in isolation, ensuring fast execution, predictable results, and no impact on other environments.



Purpose

To define a separate configuration for the testing environment so that tests run against in-memory databases, mocked services, and safe test data.

1. TEST PROFILE CONFIGURATION



Activate Test Profile

```
@ActiveProfiles("test")
@SpringBootTest
public class CustomerServiceTest {
    // test cases
}
```



application-test.yml

```
spring:
  datasource:
    url: jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1
    driver-class-name: org.h2.Driver
    username: sa
    password:
  jpa:
    hibernate:
      ddl-auto: create-drop
      show-sql: true
  server:
    port: 0
  logging:
    level:
      root: INFO
```



Profile Activation Sources

- @ActiveProfiles("test")
- spring.profiles.active=test
- --spring.profiles.active=test
- Maven/Gradle test configuration

5. PROFILE HIERARCHY (PRECEDENCE)



Higher precedence profiles override properties from lower precedence profiles.

2. HOW TEST PROFILE WORKS

1 Activate



Test starts with active profile "test"

2 Load Config



Spring loads application-test.yml and profile-specific beans

3 Create Context



Application context is initialized with test configuration

4 Run Tests



Tests execute using in-memory DB, mocked services, and test data

5 Tear Down



Context is closed and in-memory resources are released

3. WHAT THE TEST PROFILE PROVIDES



In-Memory Database

Uses H2 or HSQLDB for fast, isolated database operations.



Mocked Services

External services are mocked using @MockBean or test doubles.



Ephemeral Server

Server runs on random port (port: 0) to avoid port conflicts.



Test Data

Uses predefined, safe, and repeatable test datasets.



Test Logging

Appropriate log levels to keep test output clean and readable.

6. EXAMPLE - TEST-SPECIFIC BEANS

Test Configuration Class

```
@TestConfiguration
public class TestConfig {
    @Bean
    public EmailService emailService() {
        return new MockEmailService();
    }
}
```

Mock Bean in Test

```
@SpringBootTest
@ActiveProfiles("test")
public class OrderServiceTest {
    @MockBean
    private PaymentGateway paymentGateway;
}
```

4. BENEFITS



Fast

In-memory DB and mocked services reduce test execution time.



Isolated

Tests run in isolation without affecting development or production data.



Reliable

Consistent and repeatable test results every time.



Safe

No risk of modifying real data or calling real external services.



CI/CD Friendly

Optimized for automated test pipelines and parallel execution.

7. BEST PRACTICES

- ✓ Use separate application-test.yml for test configuration.
- ✓ Use in-memory databases (H2, HSQLDB) for tests.
- ✓ Mock external services using @MockBean.
- ✓ Use @ActiveProfiles("test") in test classes.
- ✓ Keep test data minimal and deterministic.
- ✓ Clean up data between tests (use @Transactional or @DirtiesContext when needed).
- ✓ Ensure tests are independent and can run in any order.

PRODUCTION PROFILE

Used in the live environment -- optimized for performance, security, and reliability.

Example: application-prod.yml

```
spring:
  profiles:
    active: prod
  datasource:
    url: jdbc:postgresql://prod-db.internal:5432/bankdb
    username: bankapp
    password: ${DB_PASSWORD}
    hikari:
      maximum-pool-size: 20
  jpa:
    hibernate:
      ddl-auto: validate
    show-sql: false
logging:
  level:
    root: WARN
```

KEY CHARACTERISTICS

- Uses real, highly available databases and external systems.
- Hardened security settings and sensitive data management.
- Performance optimized with connection pooling, caching, and tuning.
- Minimal logging (INFO/WARN/ERROR) for performance.
- Designed for stability, scalability, and high availability.

HOW TO ACTIVATE

-Dspring.profiles.active=prod

SPRING_PROFILES_ACTIVE=prod

KEY TAKEAWAY The production profile should be locked down, monitored, and thoroughly tested before deployment.

Production Profile

Secure, Scalable, and Reliable Configuration for Live Environments



The **Production Profile** provides optimized and secure configuration for the live environment, ensuring performance, availability, monitoring, and data protection.



Purpose

To define a dedicated configuration for the production environment that ensures security, stability, scalability, observability, and reliability for real users and real data.

1. PRODUCTION PROFILE CONFIGURATION



Activate Production Profile

```
@SpringBootApplication
@Profile("prod")
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```



application-prod.yml

```
spring:
  datasource:
    url: jdbc:postgresql://prod-db.company.com:5432/appdb
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
    hikari:
      maximum-pool-size: 20
      minimum-idle: 5
  jpa:
    hibernate:
      ddl-auto: validate
      show-sql: false
  server:
    port: 8080
  logging:
    level:
      root: WARN
      com.company: INFO
```



Profile Activation Sources

- @ActiveProfiles("prod")
- spring.profiles.active=prod
- --spring.profiles.active=prod
- OS Environment Variable: SPRING_PROFILES_ACTIVE=prod
- Docker / Kubernetes environment variables

5. PROFILE HIERARCHY (PRECEDENCE)



Higher precedence profiles override properties from lower precedence profiles.

2. HOW PRODUCTION PROFILE WORKS

1 Activate



Application starts with active profile "prod"

2 Load Config



Spring loads application-prod.yml and profile-specific beans

3 Create Context



Application context is initialized with production configuration

4 Run Application



Application runs with optimized settings, real services, and external resources

5 Monitor & Operate



Application is monitored, logged, and scaled in production

3. WHAT THE PRODUCTION PROFILE PROVIDES



External Databases

Uses production-grade databases (PostgreSQL, Oracle, MySQL, etc.).



Externalized Config

All sensitive properties managed via environment variables or secrets (Vault, AWS Secrets).



Security Hardened

Enforces HTTPS, strong passwords, secure headers, and access controls.



Optimized Resources

Connection pooling, caching, threading, and memory tuned for load.



Monitoring & Logging

Structured logging, metrics, tracing, and health checks enabled.

6. EXAMPLE - PRODUCTION BEAN CONFIGURATION

Production DataSource Bean

```
@Configuration
@Profile("prod")
public class DataSourceConfig {
    @Bean
    @ConfigurationProperties(prefix = "spring.datasource")
    public DataSource dataSource() {
        return DataSourceBuilder.create().build();
    }
}
```

Production Properties via Environment Variables

```
# Environment Variables
DB_URL=jdbc:postgresql://prod-db.company.com:5432/appdb
DB_USERNAME=appuser
DB_PASSWORD=*****
SPRING_PROFILES_ACTIVE=prod
SERVER_PORT=8080
LOG_LEVEL_ROOT=WARN
```

4. BENEFITS



Secure

Sensitive data is externalized and protected. HTTPS, strong headers, and secure defaults.



Reliable

Optimized connection pools, timeouts, and health checks ensure high availability.



High Performance

Tuned resources, caching, and efficient logging improve performance.



Observable

Health endpoints, metrics, and centralized logging enable full visibility.



Scalable

Designed to scale with load using external services and stateless configuration.

7. BEST PRACTICES

- ✓ Use application-prod.yml for all production-specific settings.
- ✓ Externalize all sensitive information.
- ✓ Never commit secrets to source control.
- ✓ Use connection pooling and set proper timeouts.
- ✓ Set logging to WARN or ERROR in production.
- ✓ Enable health, metrics, and distributed tracing.
- ✓ Test configuration in a staging environment first.
- ✓ Regularly rotate credentials and secrets.
- ✓ Backup, monitor, and review configurations continuously.

KEY TAKEAWAYS



Production profile isolates live configuration from all other environments.



Keep configuration secure, externalized, and environment-specific.



Optimize for performance, availability, and observability in production.



Follow best practices to ensure a stable, secure, and scalable production system.

TRY IT YOURSELF — EXERCISE 1: PROFILE PURPOSES

Analysis exercise -- state why dev, test, and prod exist for Northstar CRM, and why a blank prod password is unacceptable.

STEP-BY-STEP

STEP	WHAT TO DO
1. Write goals	In notes/profiles.md, write one sentence each for dev, test, and prod.
2. Check reference	Align your three sentences with the reference table below before moving on.
3. Incident story	Explain in writing why a blank prod password committed in YAML is unacceptable.
4. Fixtures	Confirm that under dev, CUS-1001 and CUS-1002 must still be callable.

Reference: dev / test / prod goals

```
dev    -> Local H2-friendly / verbose-safe settings
test   -> Deterministic automated tests
prod   -> Fail-fast; secrets from environment
```

PASS CRITERIA

- Three profiles described (dev, test, prod).
- YAML-secret anti-pattern called out.
- Dev fixtures (CUS-1001 / CUS-1002) mentioned.

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 1 before continuing: exercises/exercise-01-profile-purposes.md.

TRY IT YOURSELF — EXERCISE 2: PROFILE YAML TODOS (STARTER)

Hands-on starter -- fill every ____ and // TODO in three profile YAML sketches without adding real secrets.

STEP-BY-STEP

STEP	WHAT TO DO
1. Create files	Create notes/application.yml, notes/application-dev.yml, notes/application-prod.yml.
2. Fill TODOs	Replace every ____ in the base, dev, and prod sketches using the hints on the right.
3. Self-check	Confirm the prod file has no literal password strings anywhere.
4. Reflect	Record the lab evidence correlation ID: lab26-001 (or lab-request-001).

Starter TODOs (replace every ____)

```
# application.yml
spring.application.name: ____
server.port: ____

# application-dev.yml
northstar.integration.api-base-url: http://localhost:____
logging.level.com.northstar: ____

# application-prod.yml -- no real passwords
northstar.integration.api-base-url: $
{NORTHSTAR_API_BASE_URL:____}
```

HINTS

name = northstar-crm, port = 8080, dev logging = DEBUG, prod default placeholder = CHANGE_ME.

KEY TAKEAWAY TRY IT NOW (STARTER) -- Complete Exercise 2 before continuing: [exercises/exercise-02-profile-yaml-todos.md](#).

ENVIRONMENT VARIABLES (1 OF 2)

External key-value pairs provided by the operating system or the deployment environment -- essential for security and portability.

Environment variables enable applications to be configured without changing any code or configuration files.

HOW IT WORKS



KEY BENEFITS

- Keep sensitive information out of code and configuration files.
- Easily change values per environment (dev, test, prod); ideal for cloud, container, and CI/CD deployments.
- Improves portability, maintainability, security, and compliance.

KEY TAKEAWAY Environment variables have higher precedence than values in application files.

ENVIRONMENT VARIABLES (2 OF 2)

Naming conventions, precedence, and how to set environment variables on different platforms.

NAMING CONVENTIONS

Property Name	Environment Variable
spring.datasource.url	SPRING_DATASOURCE_URL
spring.datasource.username	SPRING_DATASOURCE_USERNAME
server.port	SERVER_PORT
northstar.integration.timeout-ms	NORTHSTAR_INTEGRATION_TIMEOUT_MS

Uppercase letters; dots (.) and hyphens (-) in property names are replaced with underscores (_).

PRECEDENCE (HIGH TO LOW)

- 1. Command-line arguments
- 2. Environment variables
- 3. application-{profile}.yml / .properties
- 4. application.yml / .properties
- 5. Defaults in the code

EXAMPLES: SETTING ENVIRONMENT VARIABLES

```
# Linux/macOS      # Windows (PowerShell)      # Docker
export SPRING_PROFILES_ACTIVE=prod  $env:SPRING_PROFILES_ACTIVE="prod"  docker run
-e SPRING_PROFILES_ACTIVE=prod app
```

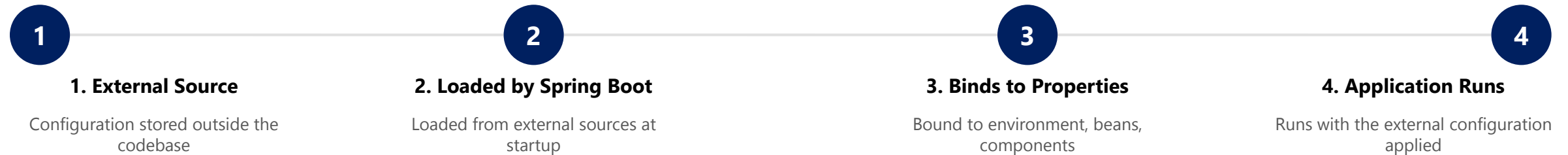
KEY TAKEAWAY Use environment variables for secrets, credentials, URLs, API keys, and environment-specific settings.

EXTERNALIZED CONFIGURATION (1 OF 2)

Storing application settings outside the application code so behavior can change without recompiling or redeploying.

Spring Boot makes it easy to externalize configuration and provides a flexible way to manage settings across environments.

HOW IT WORKS



KEY BENEFITS

- Change configuration without code changes; environment-agnostic deployments (dev, test, prod).
- Keep sensitive information out of code (security).
- Easier to manage and maintain; better portability across platforms and clouds.

KEY TAKEAWAY Externalized configuration enables flexibility, security, and scalability across all environments.

Externalized Configuration in Spring Boot

Externalized configuration lets you store configuration outside the application, so it can be changed without rebuilding the application.

Why Externalize?



Change configuration without rebuilding or redeploying.



Adapt easily across different environments.



Keep sensitive information out of source code.



Promote 12-Factor App principles.



Centralize and standardize configuration management.

How Externalized Configuration Works

1. Configuration Sources

Command Line Arguments
`--server.port=8080`

Environment Variables
`SERVER_PORT=8080`

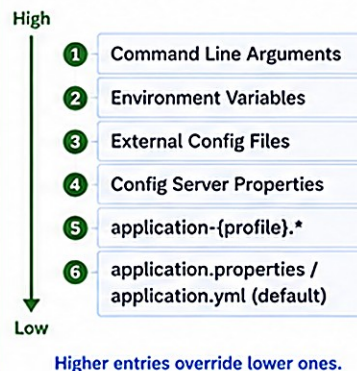
External Config Files
file:./config/application.yml

Config Server
Spring Cloud Config

Secrets Managers
AWS Secrets Manager, Azure Key Vault, etc.

2. Property Resolution

Spring Boot PropertySource Order



3. Application



Configuration injected into Beans and Components

External Systems (Optional)



Secrets Stores



Examples

Environment Variable

```
export SERVER_PORT=8080
export DB_URL=jdbc:postgresql://db:5432/app
```



OS environment variables override file properties.

External File (application.yml)

```
server:
  port: 8080
spring:
  datasource:
    url: jdbc:postgresql://db:5432/app
```



Place file outside the jar:
file:./config/application.yml

Command Line Argument

```
java -jar app.jar \
  --spring.profiles.active=prod \
  --db.url=jdbc:postgresql://db:5432/app
```



Highest priority – Overrides all other sources.

Best Practices



Never commit secrets to source control.



Use environment variables or secret stores for sensitive data.



Keep defaults in the application for developer convenience.



Document required configurations and their sources.



Use Config Server for centralized management in microservices.

EXTERNALIZED CONFIGURATION (2 OF 2)

External source examples, a before/after code comparison, and best practices for externalizing configuration.

EXAMPLES OF EXTERNAL SOURCES

- Property/YAML files: application.properties / application.yml.
- Environment variables and command-line arguments.
- Config Server (Spring Cloud Config).
- Secrets management solutions (AWS Secrets Manager, Vault, etc.).

BEST PRACTICES

Store Config Externally

Env Vars for Secrets

12-Factor Principle

Document Defaults

Before (Hard-coded)

```
@Value("${db.url}")  
private String dbUrl = "jdbc:mysql://localhost:3306/bank";
```

After (Externalized)

```
@Value("${db.url}")  
private String dbUrl; // Value comes from an external source
```

KEY TAKEAWAY Store configuration in external sources, not in code. Follow the 12-factor app principle: strict separation of config from code.

TRY IT YOURSELF — EXERCISE 3: CONFIGURATION PROPERTIES SKETCH

Documentation exercise -- sketch NorthstarIntegrationProperties fields and the prod fail-fast rule, with no real secrets.

STEP-BY-STEP

STEP	WHAT TO DO
1. Fields	In notes/northstar-props.md, list placeholder fields: apiBaseUrl, apiKey (env-only in prod), connectTimeoutMs.
2. Prefix	Propose the YAML prefix northstar.integration for the typed binding.
3. Fail-fast	Write the rule: prod must fail startup if DB_USERNAME / DB_PASSWORD / NORTHSTAR_API_KEY are missing.
4. .env.example	List placeholder keys only -- never real values.

Sketch: northstar.integration.*

```
northstar:
  integration:
    api-base-url: ${NORTHSTAR_API_BASE_URL}
    api-key: ${NORTHSTAR_API_KEY} # env-only, prod
    connect-timeout-ms: ${NORTHSTAR_TIMEOUT_MS:2000}
```

PASS CRITERIA

- Three fields listed (apiBaseUrl, apiKey, connectTimeoutMs).
- Fail-fast prod rule written.
- No real secrets anywhere in the sketch.

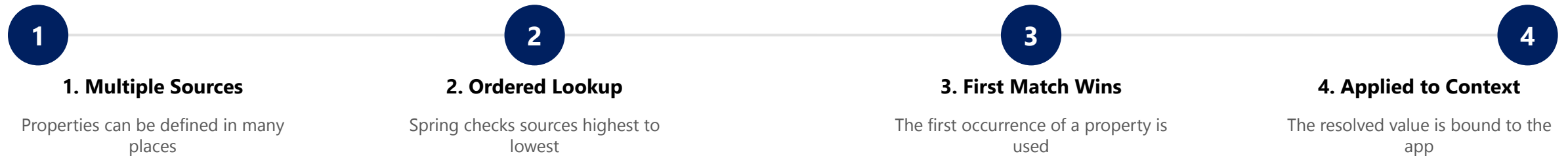
KEY TAKEAWAY TRY IT NOW -- Complete Exercise 3 before continuing: [exercises/exercise-03-config-properties-sketch.md](#).

PROPERTY RESOLUTION (1 OF 2)

Spring Boot uses a well-defined order to resolve configuration properties. When a property is defined in multiple places, the highest-precedence source wins.

Understanding this order helps you predict which value will be used and avoid unexpected overrides.

HOW PROPERTY RESOLUTION WORKS



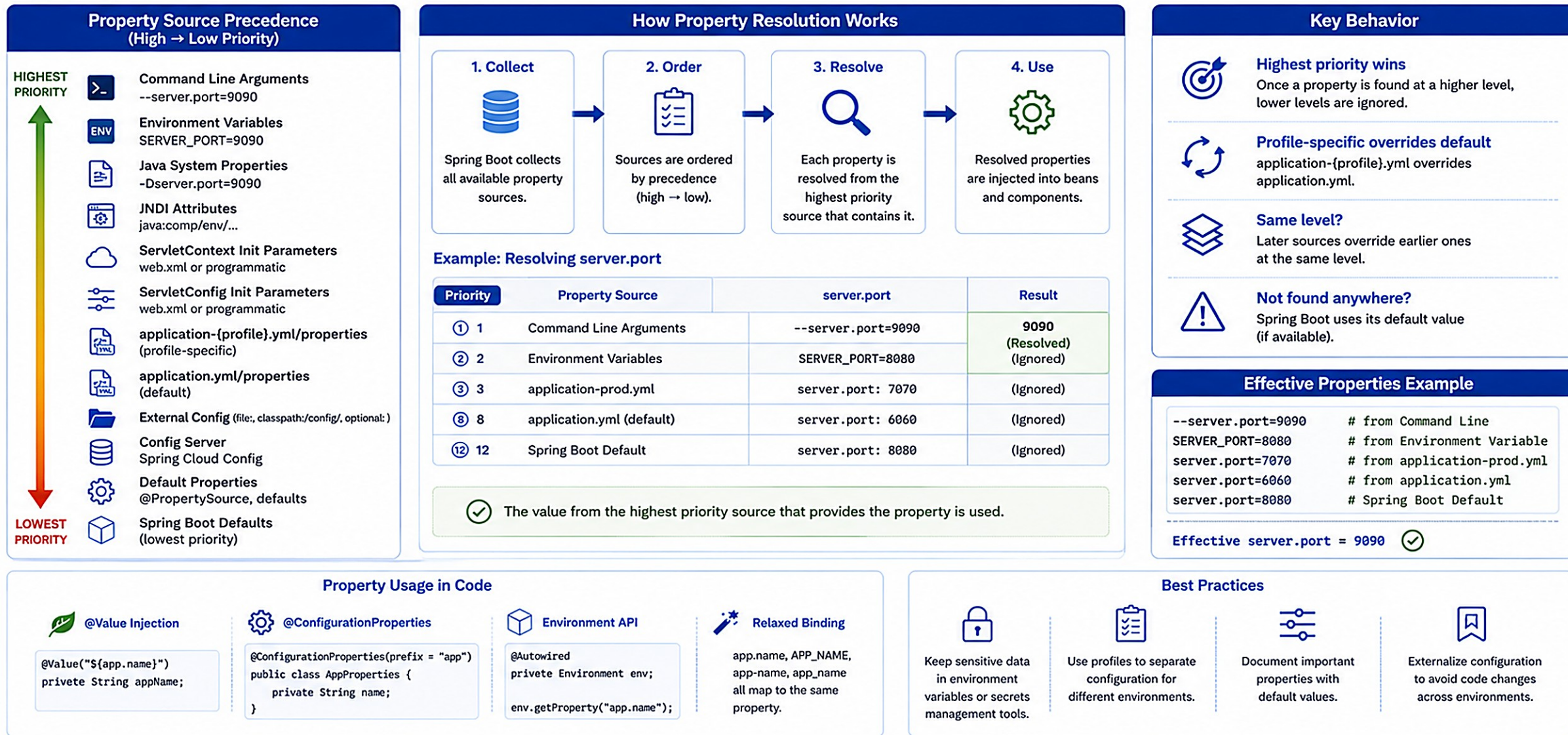
KEY TAKEAWAYS

- The highest-precedence source wins; external sources override internal files.
- Profiles and environments affect values; use this order to design predictable configuration.
- Great for troubleshooting and debugging unexpected overrides.

KEY TAKEAWAY When a property is found in a higher source, Spring ignores all lower sources for that property.

Spring Boot Property Resolution

Spring Boot resolves properties from multiple sources in a fixed order. Higher priority sources override lower priority sources.



Tip: Use Spring profiles, environment variables, and external configuration for clean, secure, and flexible configuration management.

PROPERTY RESOLUTION (2 OF 2)

The complete property source precedence order, and a worked example of the same property defined in five different places.

PROPERTY SOURCE PRECEDENCE (HIGHEST TO LOWEST)

- 1. Command-line arguments (e.g. --app.name=MyApp-CLI)
- 2. Environment variables (e.g. APP_NAME=MyApp-ENV)
- 3. Java system properties (e.g. -Dapp.name=MyApp-SYS)
- 4. JNDI attributes from java:comp/env
- 5. ServletConfig / ServletContext init parameters
- 6. application-{profile}.yml / .properties (profile-specific files)
- 7. application.yml / .properties (base file)
- 8. @PropertySource files
- 9. Default properties (via setDefaultProperties) -- lowest precedence

EXAMPLE: SAME PROPERTY, MULTIPLE PLACES

Source (High to Low)	app.name Value
Command-line argument	MyApp-CLI
Environment variable	MyApp-ENV
application-prod.yml	MyApp-ProdYML
application.yml	MyApp-YML
Default in @Value / code	MyApp-Default

If app.name is passed as a command-line argument, 'MyApp-CLI' wins and every lower source is ignored for that key.

KEY TAKEAWAY Prefer external configuration for flexibility and security; test with different sources to ensure correct behavior.

TRY IT YOURSELF — EXERCISE 4: PROPERTY OVERRIDE ORDER

Architecture exercise -- rank CLI, env, profile YAML, and base YAML by precedence, then write both activation styles.

STEP-BY-STEP

STEP	WHAT TO DO
1. Rank	In notes/override-order.md, number the four sources highest to lowest.
2. Check reference	Compare your ranking to the reference table on the right; correct mistakes.
3. Activation pair	Write example activations: -Dspring.profiles.active=dev and SPRING_PROFILES_ACTIVE=prod.
4. Measurement plan	Note that Lab 26 asks for measured override evidence -- defer capturing it to the lab.

Reference: precedence (highest first)

```
1. Command-line args / -D
2. Environment variables
3. Profile-specific YAML
4. Base application.yml
```

PASS CRITERIA

- Order matches the reference table.
- Both activation styles listed.
- Lab measurement deferred explicitly.

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 4 before continuing: exercises/exercise-04-override-order.md.

MANAGING SENSITIVE INFORMATION (1 OF 2)

Passwords, API keys, tokens, and database credentials must be protected -- never hard-coded in the application.

Spring Boot provides multiple ways to externalize and secure sensitive configuration so it is never hard-coded in the application.

WHY IT MATTERS



Protects Access

Protects against unauthorized access and data breaches.



Compliance

Helps meet security and compliance requirements.



Safe Sharing

Enables safe sharing and deployment across environments.



Reduced Exposure

Reduces risk of accidental exposure in code or logs.

KEY TAKEAWAYS

- Never store secrets in source code or configuration files in version control.
- Use externalized configuration and secret management tools; limit access to secrets and rotate them regularly.
- Audit usage and avoid logging sensitive information.

KEY TAKEAWAY Never store secrets in source code or configuration files in version control.

MANAGING SENSITIVE INFORMATION (2 OF 2)

Concrete ways to manage sensitive information, best practices, and a worked application.yml example.

WAYS TO MANAGE SENSITIVE INFORMATION

Environment Variables — set secrets as environment variables; Spring Boot reads them at runtime.

Config Server + Vault — store secrets in HashiCorp Vault and access via Spring Cloud Config.

Cloud Secret Managers — AWS Secrets Manager, Azure Key Vault, Google Secret Manager.

External Files (Not in Repo) — store secrets in external files mounted at runtime, never in the repo.

Encrypted Properties — encrypt values and decrypt at runtime using Jasypt or similar libraries.

Example: application.yml

```
spring:
  datasource:
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD} # from environment
```

BEST PRACTICES

- Externalize all secrets.
- Never commit .env (.gitignore it).
- Grant least-privilege access.
- Rotate secrets regularly.
- Never log secret values.

GITIGNORE + .ENV.EXAMPLE PATTERN

.env.example (committed, placeholders only)

```
DB_USERNAME=changeme
DB_PASSWORD=changeme
NORTHSTAR_API_KEY=changeme

# .env itself is .gitignore'd -- never commit it
```

KEY TAKEAWAY Treat secrets like keys to your house -- store them securely, control access, and change them regularly.

SECRETS MANAGEMENT BASICS (1 OF 2)

Secrets are sensitive pieces of information such as passwords, API keys, tokens, and certificates -- managing them securely is critical.

A secrets manager provides a secure, centralized place to store secrets and control access to them.

WHY USE A SECRETS MANAGER?



Centralized Storage

One secure, centralized place to store secrets.



Fine-grained Access

Fine-grained access control per secret and consumer.



Automatic Rotation

Automatic secret rotation without code changes.



Audit Logging

Audit logging and monitoring of secret access.

COMMON SECRETS

Database Passwords

API Keys & Tokens

Email Credentials

Encryption Keys/Certs

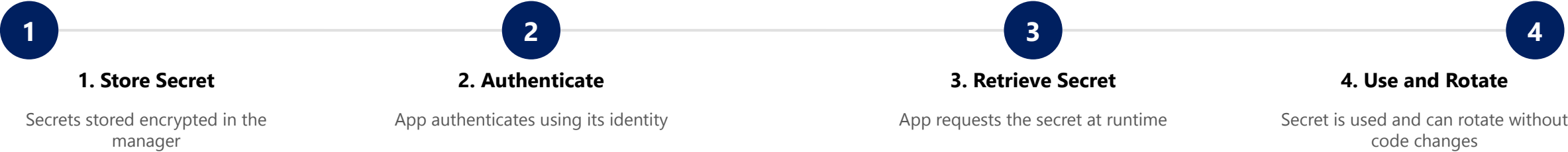
Third-Party Credentials

KEY TAKEAWAY A secrets manager provides a secure, centralized place to store secrets and controls access to them.

SECRETS MANAGEMENT BASICS (2 OF 2)

How a secrets manager works end to end, popular secrets managers, and a worked Spring Boot example.

HOW IT WORKS



POPULAR SECRETS MANAGERS

Manager	Description
AWS Secrets Manager	Fully managed, integrates with IAM and AWS services.
Azure Key Vault	Secure store for secrets, keys, and certificates in Azure.
Google Secret Manager	Managed service to store, manage, and access secrets.
HashiCorp Vault	Open-source and enterprise solution for secrets and encryption.

Example: Spring Boot + AWS Secrets Manager

```
spring:
  datasource:
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
aws:
  secretsmanager:
    secret-name: myapp/prod/db-credentials
    region: us-east-1
```

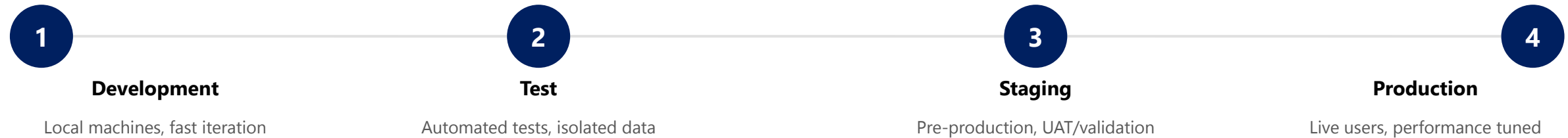
KEY TAKEAWAY Rotate secrets regularly, grant least-privilege access, and never log or expose secret values.

CONFIGURATION ACROSS ENVIRONMENTS (1 OF 2)

Different environments have different requirements. Spring Profiles and externalized configuration let us manage them cleanly and safely.

The goal is to promote consistency, reduce risk, and make deployments reliable -- keep code identical, change only the configuration.

THE BIG PICTURE



KEY TAKEAWAYS

- Keep code identical; change configuration, not code.
- Use profiles to separate environment concerns; store sensitive information securely.
- Externalize configuration for portability and scalability; test in lower environments before promoting to production.

KEY TAKEAWAY Use Spring Profiles and externalized configuration to keep code the same and settings environment-specific.

Configuration Across Environments

Use Spring Profiles and externalized configuration to manage environment-specific settings consistently across Development, Test, and Production.

**Goal**
Same application, different configuration for each environment.

**1. DEVELOPMENT (dev)**
For local development

**Purpose**
Fast development, debugging, local tools and mocks

**Active Profile** dev

**Key Characteristics**

- Debug logging
- Local database (H2 / MySQL)
- External services mocked
- Relaxed security
- Developer conveniences enabled

```
application-dev.yml
server:
  port: 8080
logging:
  level:
    root: DEBUG
datasource:
  url: jdbc:h2:mem:devdb
feature:
  beta-enabled: true
```

**2. TEST (test)**
For automated & manual testing

**Purpose**
Validate functionality, integrations and performance

**Active Profile** test

**Key Characteristics**

- Info logging
- Test database (isolated)
- Real or mock integrations
- Test security setup
- Seeded test data

```
application-test.yml
server:
  port: 8081
logging:
  level:
    root: INFO
datasource:
  url: jdbc:postgresql://test-db:5432/app_test
feature:
  beta-enabled: false
```

**3. PRODUCTION (prod)**
For live users and operations

**Purpose**
Stability, security, performance and reliability

**Active Profile** prod

**Key Characteristics**

- Warn / Error logging
- Production database (HA)
- Real external services
- Strict security
- Performance optimizations

```
application-prod.yml
server:
  port: 80
logging:
  level:
    root: WARN
datasource:
  url: jdbc:postgresql://prod-db:5432/app
feature:
  beta-enabled: false
```

How It Works

**1. Package Once**
Build the application artifact once (JAR/WAR/Docker image).

**2. Externalize Configuration**
Environment-specific properties are externalized.

**3. Activate Profile**
Activate the appropriate profile at runtime for each environment.

**4. Run with Environment Settings**
The application runs using the correct configuration for that environment.

Example: Property Differences				
Property	Development (dev)	Test (test)	Production (prod)	Description
 server.port	8080	8081	80	Port exposed by the application
 logging.level.root	DEBUG	INFO	WARN	Logging level
 datasource.url	jdbc:h2:mem:devdb	jdbc:postgresql://test-db:5432/app_test	jdbc:postgresql://prod-db:5432/app	Database connection URL
 feature.beta-enabled	true	false	false	Feature toggle example

**Command Line**

```
--spring.profiles.active=dev
java -jar app.jar \
--spring.profiles.active=prod
```

**Environment Variable**

```
SPRING_PROFILES_ACTIVE=dev
export SPRING_PROFILES_ACTIVE=prod
```

**application.properties / application.yml**

```
spring.profiles.active=dev
```

**Config Server / Orchestrator**
Set profile via Config Server, Kubernetes, Docker, etc.

CONFIGURATION ACROSS ENVIRONMENTS (2 OF 2)

A worked database configuration example across all four environments, file naming conventions, and how to activate a profile.

EXAMPLE: DATABASE CONFIGURATION

Property	development	test	staging	production
spring.datasource.url	localhost:5432/bank	test-db:5432/bank	staging-db:5432/bank	prod-db.cluster:5432/bank
spring.datasource.username	dev_user	test_user	staging_user	prod_user
spring.datasource.password	dev_pass	test_pass	staging_pass	(from secret store)
logging.level.root	DEBUG	INFO	INFO	WARN

PROFILE-SPECIFIC FILE NAMING

- application.yml -- default (all environments)
- application-dev.yml -- Development
- application-test.yml -- Test
- application-staging.yml -- Staging
- application-prod.yml -- Production

WAYS TO ACTIVATE A PROFILE

Command Line

Environment Variable

application.yml

IDE Run Configuration

KEY TAKEAWAY Right configuration in the right environment leads to secure, reliable, and scalable applications.

TRY IT YOURSELF — EXERCISE 5: ACTIVATION COMMAND DRILL

Analysis exercise -- write Windows and macOS profile activation commands for your notes.

STEP-BY-STEP

STEP	WHAT TO DO
1. -D form	In notes/activation-commands.md, write the -Dspring.profiles.active=dev style (or the mvn equivalent).
2. Env form	Write both the Windows PowerShell and macOS/Linux environment-variable forms.
3. Prod caution	Note: never run the prod profile without required env vars set -- expect fail-fast.
4. Boundary	Do not start the full Lab 26 app in this exercise unless the instructor asks.

Windows vs macOS activation

```
# -D form (via Maven)
mvn spring-boot:run -Dspring-boot.run.arguments=
  --spring.profiles.active=dev

# Windows PowerShell
$env:SPRING_PROFILES_ACTIVE="test"

# macOS / Linux
export SPRING_PROFILES_ACTIVE=test
```

PASS CRITERIA

- -D example present.
- Env-var example present (both OS).
- Prod fail-fast caution written.

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 5 before continuing: exercises/exercise-05-activation-drill.md.

CONFIGURATION BEST PRACTICES (1 OF 2)

Following best practices ensures your configuration is secure, maintainable, portable, and easy to manage across different environments.

FOLLOW THESE CORE PRINCIPLES

**Externalize**
Keep configuration outside the code.

**Organize**
Use a clear structure and naming conventions.

**Secure**
Protect secrets and sensitive data.

**Environment-Aware**
Use profiles and variables for environment-specific config.

**Consistent**
Maintain consistency across all environments and services.

BEST PRACTICES FOR SPRING CONFIGURATION (1-5)

#	Practice	What It Means
1	Externalize Everything	Move all configurable values out of the code.
2	Use the Right Source	Prefer environment variables, Config Server, or Vault for sensitive values.
3	Follow Naming Conventions	Use meaningful property names (e.g. app.datasource.url).
4	Use Profiles Wisely	Keep common settings in application.yml and override only what's needed.
5	Separate Config by Concern	Group properties by feature or module.

KEY TAKEAWAY Good configuration is the foundation of a resilient, secure, and scalable application.

CONFIGURATION BEST PRACTICES (2 OF 2)

Practices 6-10, what to avoid, and an example of a good configuration file structure.

BEST PRACTICES FOR SPRING CONFIGURATION (6-10)

- 6. Never Commit Secrets** — use .gitignore and secret management tools.
- 7. Use Placeholders and Defaults** — provide sensible defaults, but never for secrets; use \${...:default}.
- 8. Validate Configuration** — fail fast for missing or invalid critical properties.
- 9. Document Configuration** — maintain clear documentation for required properties and their meaning.
- 10. Monitor and Audit** — track changes to configuration and access to secrets.

WHAT TO AVOID

Hard-coding Values

Plain-text Secrets

One Config for All Envs

Skipping Validation

Example: Good Config Structure

```
src/main/resources/
├── application.yml           # common
├── application-dev.yml      # development
├── application-test.yml     # test
├── application-staging.yml  # staging
├── application-prod.yml     # production
└── application-secrets.yml  # local, .gitignored

# Keep common properties in application.yml
# and override only what changes per profile.
```

KEY TAKEAWAY Externalize everything, never commit secrets, and validate configuration at startup -- fail fast for missing critical properties.

TRY IT YOURSELF — EXERCISE 6: LAB 26 READINESS (CAPSTONE)

Documentation capstone -- ready the lab26-crm path and secret hygiene before starting the graded Lab 26.

STEP-BY-STEP

STEP	WHAT TO DO
1. Depends on	In notes/lab26-readiness.md, note the Lab 25 layered CRM dependency.
2. Path	Write the project path: examples/lab26-crm.
3. Hygiene	Confirm .gitignore will cover .env and target/.
4. Defer	Note that transactions (Lab 27) and security (Lab 28) wait -- do not build them yet.

.gitignore hygiene check

```
# Must be covered before Lab 26 starts
.env
target/

# Committed instead (placeholders only)
.env.example
```

PASS CRITERIA

- Lab 25 dependency noted.
- Path written correctly (examples/lab26-crm).
- .env / target/ hygiene stated.

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 6 (capstone) before Lab 26: exercises/exercise-06-lab26-readiness.md.

LAB OVERVIEW -- SPRING PROFILES AND CONFIGURATION

Lab 26: Spring Profiles and Configuration -- Northstar CRM Environments

BUSINESS SCENARIO

- Incidents from dev settings leaking into production (open H2 console, blank DB password, verbose SQL) are unacceptable.
- Leadership freezes: running configuration depends on where the app is deployed; production credentials arrive only via environment variables.
- Missing required prod properties must fail fast at startup -- never connect with blank passwords.

LEARNING OBJECTIVES

- Explain Spring's property-source override order: CLI > env > application-{profile}.yaml > application.yaml > code defaults.
- Convert application.properties to application.yaml and structure dev/test/prod profile files.
- Activate a profile with -Dspring.profiles.active and with SPRING_PROFILES_ACTIVE.
- Bind externalized configuration with @ConfigurationProperties instead of scattered @Value.
- Apply secrets-handling practices while keeping CRM fixtures working under dev.

WHAT YOU BUILD

- Copy lab25-crm to lab26-crm; convert to application.yaml with dev/test/prod profile files; activate profiles two ways; prove CLI > env > profile-YAML override order with northstar.integration.timeout-ms; add a validated NorthstarIntegrationProperties; ship .env.example with placeholders only; confirm dual green mvn test under test and a CUS-1001 smoke test under dev.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) must still work under dev; correlation ID lab26-001 anchors every verification.

LAB TASKS AND DELIVERABLES

Northstar CRM Environments -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Branch lab25-crm to lab26-crm; inventory existing properties before deleting anything.
2	Convert shared defaults to application.yml (name, port, management, logging).
3	Author application-dev.yml and application-test.yml (H2, logging levels, mocks).
4	Author application-prod.yml with env-only secrets -- no default passwords.
5	Activate profiles two ways: -Dspring-boot.run.profiles and SPRING_PROFILES_ACTIVE.
6	Prove override order with northstar.integration.timeout-ms (YAML -> env -> CLI).
7	Bind NorthstarIntegrationProperties (@Validated) and ship .env.example.
8	Tests under test profile + CUS-1001/CUS-1002 smoke under dev; failure experiments + secrets hygiene.

EXPECTED DELIVERABLES

- application.yml + dev/test/prod profile files.
- NorthstarIntegrationProperties + enable config.
- .env.example placeholders only.
- Evidence of -D and env profile activation; fail-fast prod startup evidence.
- Override-order notes with measurements; dual green tests under test; no secrets or target/ committed.

RUBRIC (100 MARKS)

- Environment/Structure 10 · Core Impl 30 · Activation/Override 15 · Externalized/Fail-fast 15 · Failure Handling 10 · Security/Secrets 10 · Docs 10

KEY TAKEAWAY Real secrets in Git are an honor violation; prod YAML with default passwords loses security/fail-fast marks; no override evidence loses activation marks.

MODULE 26 SUMMARY

In this module, we explored how to build flexible, secure, and environment-aware Spring Boot applications using Spring Profiles and externalized configuration.

KEY CONCEPTS COVERED



Properties & YAML

application.properties and application.yml, and when to use each.



Spring Profiles

Define, activate, and structure environment-specific configuration.



Externalized Config

Environment variables, CLI args, and property source precedence.



Property Resolution

The full precedence order Spring uses to resolve conflicting values.



Secrets Management

Never hard-code secrets; environment variables and secrets managers.



Best Practices

Externalize, organize, secure, validate, and fail fast.

MODULE FLOW RECAP

1

YAML & Properties

2

Spring Profiles

3

Externalize & Resolve

4

Secure Secrets

5

Lab: Northstar CRM

KEY TAKEAWAY Good configuration leads to flexible applications. Secure configuration leads to trustworthy applications.

KNOWLEDGE CHECK (1 OF 3)

Section 1: Multiple Choice -- test your understanding of Spring Profiles, configuration files, and profile activation.

1. What is the primary purpose of Spring Profiles?

- A. To encrypt configuration files
- B. To create database backups
- C. To provide environment-specific configuration**
- D. To define user roles and permissions

3. Which of the following is a valid way to activate a profile?

- A. `spring.profiles.active=dev` (in `application.properties`)
- B. `--spring.profile=dev`
- C. `-Dspring.profile.active=dev`
- D. All of the above**

2. Which file is the default configuration file in a Spring Boot application?

- A. `config.properties`
- B. `application.properties`**
- C. `spring-config.yml`
- D. `bootstrap.properties`

4. Which of the following has the highest precedence in property resolution?

- A. `application.properties`
- B. Environment variables**
- C. `application-{profile}.yml`
- D. Default values in code

KEY TAKEAWAY Spring Profiles provide environment-specific configuration; environment variables outrank profile and base files in property resolution.

KNOWLEDGE CHECK (2 OF 3)

Two more multiple-choice questions, plus Section 2: Match the Following.

5. How can you externalize configuration in a Spring Boot application?

- A. Store values in code constants
- B. Use application.yml and environment variables
- C. Rebuild the application for every change
- D. Hardcode values in beans

6. Which of the following is NOT a recommended practice for configuration?

- A. Externalize properties
- B. Store secrets in plain text in code
- C. Use profiles for environment differences
- D. Validate configuration at startup

SECTION 2: MATCH THE FOLLOWING

Concept	Description
1. Spring Profile	b. Defines a set of properties for a specific environment.
2. Externalized Configuration	e. Keeping configuration outside of application code.
3. Environment Variable	c. External source used to override application properties.
4. Property Resolution	d. Process of Spring finding the final value of a property.
5. Secrets Manager	a. Used to store and retrieve sensitive information securely.

KEY TAKEAWAY A property must be moved out of code (externalized) before profiles, environment variables, or a secrets manager can manage it.

KNOWLEDGE CHECK (3 OF 3)

Section 3: True or False, Section 4: Scenario-Based, and Section 5: Short Answer -- with model answers.

SECTION 3: TRUE OR FALSE

Statement	Answer
Spring Profiles allow running the same app with different configs.	True
application-{profile}.properties overrides application.properties when active.	True
Environment variables cannot override properties defined in files.	False
Storing database passwords in application.properties is a best practice.	False
The order of property resolution determines the final property value.	True

SECTION 5: SHORT ANSWER

- 1. **Three best practices** — externalize configuration; never commit secrets (use env vars/secrets managers); use profiles to separate dev/test/prod settings.
- 2. **Benefits of externalized configuration** — no code changes needed per environment; secrets stay out of source control; easier portability and scalability.

SECTION 4: SCENARIO-BASED

- 1. **Different DB URLs per environment** — define shared defaults in application.yml, then override spring.datasource.url in application-dev/test/prod.yml, and activate the right profile per deployment.
- 2. **An API key that must not live in code or files** — store it as an environment variable (or in a secrets manager), reference it as \${API_KEY} with no default, and let startup fail fast if it's missing.

KEY TAKEAWAY Review these concepts and apply them in the lab -- strong configuration leads to secure, maintainable, environment-ready applications.

MODULE 26 COMPLETE

Spring Configuration, Profiles and Environments

You can now build Spring Boot applications that adapt safely across development, test, and production using profiles, externalized configuration, and secure secrets handling.